ORBIT · PYTHON · PAPER 09

第 9 套

60 题 · 完整题目与答案

单选 30 多选 15 填空 / 简答 15

本套分节

一、单项选择题（30） 二、不定项选择题（15） 三、填空题（5） 四、程序代码阅读题（5） 五、程序设计题（5）

答案与解析紧随每题；本文件用于复习核对，建议先独立作答。

一、单项选择题

01 • 单项选择

以下代码的输出是什么？

```
def add_element(element, container=[]):
    container.append(element)
    print(id(container))
    return container
list1 = add_element(1)
list2 = add_element(2)
print(id(list1) == id(list2))
```

A. True
B. False
C. 可能是True或False
D. 报错

答案 A. True

解析

02 • 单项选择

关于列表的copy()方法，以下说法正确的是：

```
original = [[1, 2], [3, 4]]
copied = original.copy()
copied[0].append(5)
```

A. original不受影响
B. original也被修改
C. 报错
D. copied是深拷贝

答案 A. original不受影响

解析

03 • 单项选择

以下代码的输出是什么？

```
t = ([1, 2], 3)
original_id = id(t[0])
t[0].append(4)
print(id(t[0]) == original_id)
```

A. True
B. False
C. 报错
D. None

答案 A. True

解析

04 • 单项选择

关于字典的fromkeys()方法，以下说法正确的是：

```
d = dict.fromkeys(['a', 'b'], [])
d['a'].append(1)
```

A. 只有d['a']被修改
B. d['a']和d['b']都被修改
C. 报错
D. 不修改任何值

答案 C. 报错

解析

05 • 单项选择

以下代码的输出是什么？

```
def func(x=[]):
    x.append(1)
    return len(x)
print(func(), func(), func())
```

A. 1 1 1
B. 1 2 3
C. 3 3 3
D. 报错

答案 B. 1 2 3

解析

06 • 单项选择

关于列表的*操作符，以下说法正确的是：

```
lst = [0] * 5
lst[0] = 1
```

A. 所有元素都变成1
B. 只有第一个元素是1
C. 报错
D. 创建5个独立的0

答案 A. 所有元素都变成1

解析

07 • 单项选择

以下代码的输出是什么？

```
matrix = [[0] * 3 for _ in range(3)]
matrix[0][0] = 1
print(sum(row[0] for row in matrix))
```

A. 1
B. 3
C. 0
D. 9

答案 B. 3

解析

08 • 单项选择

关于元组的可哈希性，以下说法正确的是：

A. 所有元组都可哈希
B. 只有包含不可变元素的元组可哈希
C. 元组不可哈希
D. 取决于元组长度

答案 A. 所有元组都可哈希

解析

09 • 单项选择

以下代码会发生什么？

```
t = ([1, 2], 3)
d = {t: 'value'}
```

A. 正常执行
B. 抛出TypeError
C. 创建空字典
D. t被转换为字符串

答案 B. 抛出TypeError

解析

10 • 单项选择

关于字符串的join()方法，以下说法正确的是：

A. 修改原字符串
B. 返回新字符串
C. 只能连接字符串列表
D. 不能使用空字符串

答案 B. 返回新字符串

解析

11 • 单项选择

以下代码的输出是什么？

```
words = ['hello', 'world']
result = ' '.join(words)
words.append('!')
print(result)
```

A. "hello world !"
B. "hello world"
C. "helloworld"
D. 报错

答案 B. "hello world"

解析

12 • 单项选择

关于列表的切片赋值，以下说法正确的是：

```
lst = [1, 2, 3, 4, 5]
lst[1:3] = [10, 20, 30]
```

A. 列表长度不变
B. 列表长度可能改变
C. 报错
D. 创建新列表

答案 A. 列表长度不变

解析

13 • 单项选择

以下代码的输出是什么？

```
lst = [1, 2, 3]
lst[1:2] = []
print(lst)
```

A. [1, 3]
B. [1, 2, 3]
C. [1, [], 3]
D. 报错

答案 B. [1, 2, 3]

解析

14 • 单项选择

关于集合的copy()方法，以下说法正确的是：

A. 创建深拷贝
B. 创建浅拷贝
C. 不创建副本
D. 返回frozenset

答案 B. 创建浅拷贝

解析

15 • 单项选择

以下代码的输出是什么？

```
s1 = {1, 2, 3}
s2 = s1.copy()
s1.add(4)
print(len(s2))
```

A. 3
B. 4
C. 0
D. 报错

答案 A. 3

解析

16 • 单项选择

Pandas中，以下哪个操作最可能返回视图？

A. df'A', 'B'
B. df['A']
C. df[df['A'] > 0]
D. df.copy()

答案 A. df'A', 'B'

解析

17 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3]})
series = df['A']
series.iloc[0] = 100
print(df['A'].iloc[0])
```

A. 1
B. 100
C. 可能是1或100
D. 报错

答案 B. 100

解析

18 • 单项选择

关于字典的clear()方法，以下说法正确的是：

A. 返回新字典
B. 清空原字典
C. 删除字典对象
D. 不修改字典

答案 A. 返回新字典

解析

19 • 单项选择

以下代码的输出是什么？

```
d1 = {'a': 1, 'b': 2}
d2 = d1
d1.clear()
print(len(d2))
```

A. 2
B. 0
C. 1
D. 报错

答案 B. 0

解析

20 • 单项选择

关于列表的sort()方法的key参数，以下说法正确的是：

A. key必须是函数
B. key可以是lambda表达式
C. key指定排序依据
D. 以上都正确

答案 B. key可以是lambda表达式

解析

21 • 单项选择

以下代码的输出是什么？

```
data = [('a', 3), ('b', 1), ('c', 2)]
data.sort(key=lambda x: x[1])
print(data[0][0])
```

A. 'a'
B. 'b'
C. 'c'
D. 1

答案 B. 'b'

解析

22 • 单项选择

关于元组的拆包，以下说法正确的是：

```
t = (1, 2, 3)
a, b, c = t
```

A. 修改了元组
B. 创建了三个新变量
C. 报错
D. a、b、c指向元组元素

答案 B. 创建了三个新变量

解析

23 • 单项选择

以下代码的输出是什么？

```
t = ([1, 2], [3, 4])
a, b = t
a.append(5)
print(t)
```

A. ([1, 2], [3, 4])
B. ([1, 2, 5], [3, 4])
C. ([1, 2], [3, 4, 5])
D. 报错

答案 A. ([1, 2], [3, 4])

解析

24 • 单项选择

关于字典推导式，以下说法正确的是：

A. 修改原字典
B. 创建新字典
C. 不能使用条件
D. 返回列表

答案 B. 创建新字典

解析

25 • 单项选择

以下代码的输出是什么？

```
original = {'a': 1, 'b': 2, 'c': 3}
new = {k: v*2 for k, v in original.items() if v > 1}
print(len(new))
```

A. 1
B. 2
C. 3
D. 0

答案 B. 2

解析

26 • 单项选择

关于列表的reverse()方法，以下说法正确的是：

A. 返回新列表
B. 修改原列表，返回None
C. 不修改列表
D. 返回迭代器

答案 B. 修改原列表，返回None

解析

27 • 单项选择

以下代码的输出是什么？

```
lst = [1, 2, 3]
result = lst.reverse()
print(result)
```

A. [3, 2, 1]
B. None
C. [1, 2, 3]
D. 报错

答案 B. None

解析

28 • 单项选择

关于集合的交集运算，以下说法正确的是：

A. s1 & s2 修改s1
B. s1.intersection(s2) 返回新集合
C. s1.intersection_update(s2) 返回新集合
D. 不能计算交集

答案 A. s1 & s2 修改s1

解析

29 • 单项选择

以下代码的输出是什么？

```
s1 = {1, 2, 3}
s2 = {2, 3, 4}
s3 = s1 & s2
print(len(s1), len(s3))
```

A. 3 2
B. 2 2
C. 3 3
D. 2 3

答案 B. 2 2

解析

30 • 单项选择

关于Pandas的loc和iloc的返回值，以下说法正确的是：

A. 总是返回副本
B. 总是返回视图
C. 可能返回视图或副本
D. 取决于inplace参数

答案 A. 总是返回副本

解析

二、不定项选择题

31 • 不定项选择

以下哪些情况会导致可变默认参数问题？

A. 默认参数是列表
B. 默认参数是字典
C. 默认参数是集合
D. 默认参数是整数

答案 A. 默认参数是列表；B. 默认参数是字典；C. 默认参数是集合

解析

32 • 不定项选择

关于浅拷贝的特点，以下说法正确的有：

A. 复制对象本身
B. 不复制嵌套对象
C. 比深拷贝快
D. 适合简单对象

答案 A. 复制对象本身；B. 不复制嵌套对象；C. 比深拷贝快；D. 适合简单对象

解析

33 • 不定项选择

以下哪些操作创建新对象？

A. `lst + [1]`
B. `lst.append(1)`
C. `s.upper()`
D. `t + (1,)`

答案 A. `lst + [1]`；C. `s.upper()`；D. `t + (1,)`

解析

34 • 不定项选择

关于元组的不可变性，以下说法正确的有：

A. 元组的长度不可变
B. 元组的元素引用不可变
C. 元组中的可变对象可以修改
D. 元组不能包含列表

答案 A. 元组的长度不可变；B. 元组的元素引用不可变；C. 元组中的可变对象可以修改

解析

35 • 不定项选择

以下哪些方法会修改原集合？

A. `add()`
B. `remove()`
C. `union()`
D. `update()`

答案 A. `add()`；B. `remove()`；D. `update()`

解析

36 • 不定项选择

关于字典的视图对象，以下说法正确的有：

A. 动态反映字典变化
B. 可以迭代
C. 不能转换为列表
D. 支持集合运算

答案 A. 动态反映字典变化；B. 可以迭代；D. 支持集合运算

解析

37 • 不定项选择

以下哪些是不可变类型？

A. frozenset
B. bytes
C. bytearray
D. tuple（包含不可变元素）

答案 A. frozenset；B. bytes；D. tuple（包含不可变元素）

解析

38 • 不定项选择

关于列表的切片，以下说法正确的有：

A. 切片创建新列表
B. 切片是浅拷贝
C. 可以用于赋值
D. 可以改变列表长度

答案 A. 切片创建新列表；B. 切片是浅拷贝；C. 可以用于赋值；D. 可以改变列表长度

解析

39 • 不定项选择

以下哪些操作不会修改原字符串？

A. replace()
B. split()
C. strip()
D. 所有字符串方法

答案 D. 所有字符串方法

解析

40 • 不定项选择

关于Pandas的inplace参数，以下说法正确的有：

A. True时修改原对象
B. False时返回新对象
C. 可能影响性能
D. 所有方法都支持

答案 A. True时修改原对象；B. False时返回新对象；C. 可能影响性能

解析

41 • 不定项选择

以下哪些类型可以作为字典的键？

A. int
B. str
C. tuple（包含不可变元素）
D. list

答案 A. int；B. str；C. tuple（包含不可变元素）

解析

42 • 不定项选择

关于函数参数传递，以下说法正确的有：

A. Python使用对象引用传递
B. 不可变对象表现像值传递
C. 可变对象表现像引用传递
D. 可以通过返回值避免修改

答案 A. Python使用对象引用传递；B. 不可变对象表现像值传递；C. 可变对象表现像引用传递；D. 可以通过返回值避免修改

解析

43 • 不定项选择

以下哪些操作会改变列表长度？

A. append()
B. extend()
C. pop()
D. sort()

答案 A. append()；B. extend()；C. pop()

解析

44 • 不定项选择

关于深拷贝的应用场景，以下说法正确的有：

A. 嵌套数据结构
B. 需要完全独立的副本
C. 避免意外修改
D. 所有情况都应使用

答案 A. 嵌套数据结构；B. 需要完全独立的副本；C. 避免意外修改

解析

45 • 不定项选择

以下哪些是Python的序列类型？

A. list
B. tuple
C. str
D. dict

答案 A. list；B. tuple；C. str

解析

三、填空题

46 • 填空 / 简答

可变对象作为函数默认参数时，会在 _____ 时创建，导致多次调用 _____ 同一对象。

答案 函数定义

解析 函数定义 ， 共享

共享

可变默认参数陷阱的核心原因：

- 默认参数在函数定义时

  函数定义时创建（只创建一次）
- 所有函数调用共享

  共享同一个默认参数对象
- 可变对象会累积修改

```
def func(lst=[]):
    # 在函数定义时创建列表
    lst.append(1)
    return lst
print(func())
# [1]
print(func())
# [1, 1] - 共享同一个列表!
```

47 • 填空 / 简答

列表的 _____ 方法返回None，而 _____ 函数返回新列表。

答案 reverse() ， reversed()

解析 列表反转的两种方式：

- reverse() ：就地反转

  就地反转，修改原列表，返回None
- reversed() ：返回反转迭代器

  反转迭代器，原列表不变

```
lst = [1, 2, 3]
lst.reverse()
# lst变为[3, 2, 1]
lst2 = [1, 2, 3]
rev = reversed(lst2)
# 返回迭代器，lst2不变
print(list(rev))
# [3, 2, 1]
```

48 • 填空 / 简答

元组包含可变对象时，元组 _____ 可哈希，因为可变对象的 _____ 可能改变。

答案 不 ， 哈希值

解析 哈希值

可变对象不能作为字典键的原因：

- 字典键必须是不可变

  不可变的
- 键必须可计算哈希值

  哈希值
- 可变对象的哈希值可能变化，导致字典内部结构混乱

```
# 错误示例
lst = [1, 2]
d = {lst: 'value'}
# TypeError: unhashable type: 'list'
# 正确示例
tup = (1, 2)
d = {tup: 'value'}
# 元组可以作为键
```

49 • 填空 / 简答

在Pandas中，避免SettingWithCopyWarning的方法是使用 _____ 或先调用 _____ 方法。

答案 loc/iloc ， copy()

解析 Pandas避免链式索引的方法：

- 使用 loc 或 iloc 进行明确的单次索引
- 使用 copy() 创建显式副本

```
# 错误：链式索引
df[df['A'] > 1]['B'] = 100
# SettingWithCopyWarning
# 正确方法1：使用loc
df.loc[df['A'] > 1, 'B'] = 100
# 正确方法2：显式复制
subset = df[df['A'] > 1].copy()
subset['B'] = 100
```

50 • 填空 / 简答

集合的 _____ 方法会修改原集合，而 _____ 运算符返回新集合。

答案 update() ， | （或

解析 （或 union() ）

集合的修改方法和非修改方法：

修改原集合：

- update() ：添加多个元素
- add() ：添加单个元素
- remove() 、 discard() ：删除元素

返回新集合：

- | （或 union() ）：并集
- & （或 intersection() ）：交集
- - （或 difference() ）：差集

```
s1 = {1, 2, 3}
s2 = {3, 4, 5}
# 修改原集合
s1.update(s2)
# s1变为{1, 2, 3, 4, 5}
# 返回新集合
s3 = s1 | s2
# 返回新集合，s1和s2不变
s4 = s1.union(s2)
# 同上
```

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
class Counter:
    def __init__(self, start=0, history=[]):
        self.value = start
        self.history = history
        self.history.append(start)
    def increment(self):
        self.value += 1
        self.history.append(self.value)
c1 = Counter(0)
c1.increment()
c2 = Counter(10)
c2.increment()
print(c1.history)
print(c2.history)
```

问题：

- (1) 两个print的输出分别是什么？
- (2) 这个类有什么问题？
- (3) 如何修复？

答案 • (1) 输出：

```
[0, 1, 10, 11]
```

- (2) history是可变默认参数，所有实例共享同一个列表
- (3) 修复方法：

```
class Counter:
    def __init__(self, start=0, history=None):
        self.value = start
        self.history = history if history is not None else []
        self.history.append(start)
```

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
def merge_dicts(d1, d2, modify_original=False):
    if modify_original:
        d1.update(d2)
        return d1
    else:
        result = d1.copy()
        result.update(d2)
        return result
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 3, 'c': 4}
result1 = merge_dicts(dict1, dict2, False)
```

```
result2 = merge_dicts(dict1, dict2, True)
print(dict1)
print(result1)
print(result2)
```

问题：

- (1) 三个print的输出分别是什么？
- (2) 为什么result1和result2不同？
- (3) 这种设计的优缺点是什么？

答案 • (1) 输出：

```
{'a': 1, 'b': 3, 'c': 4}
{'a': 1, 'b': 2}
{'a': 1, 'b': 3, 'c': 4}
```

- (2) result1使用copy()创建副本，result2直接修改原字典
- (3) 优点：提供了灵活性，可选择是否修改原对象。缺点：容易出错，需要用户明确理解行为

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3, 4, 5],
    'B': [10, 20, 30, 40, 50]
})
# 方法1：链式索引
df[df['A'] > 2]['B'] = 999
# 方法2：使用loc
df.loc[df['A'] > 2, 'B'] = 888
print(df['B'].tolist())
```

问题：

- (1) 输出是什么？
- (2) 为什么方法1不生效？
- (3) 什么是链式索引？

答案 链式索引vs loc

- (1) 输出： [10, 20, 888, 888, 888]
- (2) 方法1是链式索引，产生临时副本，修改无效
- (3) 链式索引是连续的索引操作（如 df[condition]['column'] ），可能在副本上操作

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
def create_adders(n):
    adders = []
    for i in range(n):
        def adder(x):
            return x + i
```

```
        adders.append(adder)
    return adders
funcs = create_adders(3)
results = [f(10) for f in funcs]
print(results)
# 修复版本
def create_adders_fixed(n):
    adders = []
    for i in range(n):
        def adder(x, i=i):
            return x + i
        adders.append(adder)
    return adders
funcs_fixed = create_adders_fixed(3)
results_fixed = [f(10) for f in funcs_fixed]
print(results_fixed)
```

问题：

- (1) 两个print的输出分别是什么？
- (2) 第一个版本的问题是什么？
- (3) 为什么使用默认参数可以解决？

答案 • (1) 输出：

```
[12, 12, 12]
[10, 11, 12]
```

- (2) 所有函数共享同一个变量i，循环结束后i=2
- (3) 默认参数在函数定义时求值，捕获当时的i值

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
class DataContainer:
    def __init__(self, data):
        self._data = data
    @property
    def data(self):
        return self._data
    def process(self, func):
        self._data = func(self._data)
        return self
container = DataContainer([1, 2, 3])
result = container.process(lambda x: x + [4])
print(container.data)
print(result.data)
print(container is result)
```

问题：

- (1) 三个print的输出分别是什么？
- (2) 这种设计模式叫什么？

- (3) 有什么潜在问题？

答案 链式操作返回self

- (1) 输出：

```
[1, 2, 3, 4]
True
```

- (2) 流式接口（Fluent Interface）或方法链模式
- (3) 潜在问题：修改了原对象，可能造成意外副作用

解析

五、程序设计题

56 • 填空 / 简答

实现一个通用的不可变数据类：

1. ImmutableData类：
    - 初始化后不可修改
    - 支持任意属性
    - 实现 __hash__()
2. 功能要求

功能要求：

- 使用 __slots__ 优化内存
- 提供 replace() 方法创建修改版本
- 支持嵌套不可变对象

3. 应用示例

应用示例：

- 配置对象
- 数据传输对象
- 缓存键

答案

```
from typing import Any, Dict
class ImmutableData:
    """
    通用不可变数据类
    特性：
    1. 使用__slots__优化内存
    2. 初始化后完全不可修改
    3. 可哈希，能作为字典键
    4. 提供replace()方法创建修改版本
    使用示例：
    person = ImmutableData(name="Alice", age=30, city="Beijing")
    updated = person.replace(age=31)
    """
    __slots__ = ('_data', '_hash')
    def __init__(self, **kwargs):
        """
        初始化不可变数据对象
        参数：
```

```
        - **kwargs: 任意属性键值对
        """
        # 将所有参数转换为不可变形式
        immutable_data = {}
        for key, value in kwargs.items():
            immutable_data[key] = self._make_immutable(value)
        # 使用object.__setattr__绕过__setattr__限制
        object.__setattr__(self, '_data', immutable_data)
        object.__setattr__(self, '_hash', None)
    def _make_immutable(self, value):
        """递归转换为不可变对象"""
        if isinstance(value, dict):
        return ImmutableData(**value)
    elif isinstance(value, list):
        return tuple(self._make_immutable(v) for v in value)
    elif isinstance(value, set):
        return frozenset(self._make_immutable(v) for v in value)
    elif isinstance(value, ImmutableData):
        return value
    else:
        return value
def __getattr__(self, name):
    """属性访问"""
    if name.startswith('_'):
        raise AttributeError(f"'{type(self).__name__}' object has no attribute '{name}
    try:
        return self._data[name]
    except KeyError:
        raise AttributeError(f"'{type(self).__name__}' object has no attribute '{name}
def __setattr__(self, name, value):
    """禁止设置属性"""
    raise AttributeError("Cannot modify immutable object")
def __delattr__(self, name):
    """禁止删除属性"""
    raise AttributeError("Cannot modify immutable object")
def __hash__(self):
    """计算哈希值"""
    if self._hash is None:
        items = tuple(sorted(self._data.items()))
        hashable_items = self._to_hashable(items)
        object.__setattr__(self, '_hash', hash(hashable_items))
    return self._hash
def _to_hashable(self, obj):
    """转换为可哈希形式"""
    if isinstance(obj, ImmutableData):
        return self._to_hashable(tuple(sorted(obj._data.items())))
    elif isinstance(obj, (list, tuple)):
        return tuple(self._to_hashable(item) for item in obj)
```

```
    elif isinstance(obj, dict):
        return tuple(sorted((k, self._to_hashable(v)) for k, v in obj.items()))
    else:
        return obj
def __eq__(self, other):
    """相等性比较"""
    if not isinstance(other, ImmutableData):
        return False
    return self._data == other._data
def __repr__(self):
    """字符串表示"""
    items = ', '.join(f"{k}={repr(v)}" for k, v in self._data.items())
    return f"ImmutableData({items})"
def replace(self, **changes):
    """
    创建修改版本
    参数:
    - **changes: 要修改的属性
    返回:
    - ImmutableData: 新的不可变对象
    """
    new_data = self._data.copy()
    new_data.update(changes)
    return ImmutableData(**new_data)
def to_dict(self):
    """转换为字典"""
    result = {}
    for key, value in self._data.items():
        if isinstance(value, ImmutableData):
            result[key] = value.to_dict()
        elif isinstance(value, tuple):
            result[key] = list(value)
        elif isinstance(value, frozenset):
                result[key] = set(value)
            else:
                result[key] = value
        return result
# === 使用示例 ===
print("=== 不可变数据类示例 ===\n")
# 示例1：创建配置对象
config = ImmutableData(
    app_name="MyApp",
    version="1.0.0",
    database={"host": "localhost", "port": 5432},
    features=["auth", "api", "cache"]
)
print(f"配置: {config}")
print(f"应用名称: {config.app_name}")
```

```
print(f"数据库: {config.database}")
print(f"功能: {config.features}")
# 示例2：尝试修改（会失败）
try:
    config.app_name = "NewApp"
except AttributeError as e:
    print(f"\n   修改被阻止: {e}")
# 示例3：使用replace创建新版本
updated_config = config.replace(version="2.0.0")
print(f"\n更新版本后: {updated_config}")
print(f"原配置不变: {config.version}")
# 示例4：作为字典键
cache = {config: "cached_result"}
print(f"\n作为字典键: {len(cache)} 个条目")
# 示例5：数据传输对象
person = ImmutableData(
    name="Alice",
    age=30,
    address={"city": "Beijing", "district": "Haidian"}
)
print(f"\n人员信息: {person}")
print(f"地址: {person.address}")
print(f"转为字典: {person.to_dict()}")
```

解析

57 • 填空 / 简答

实现检测和分析对象可变性的工具：

1. 检测函数

检测函数：

- is_mutable(obj)
- find_mutable_parts(obj)
- mutability_report(obj)

2. 分析功能

分析功能：

- 递归检查嵌套结构
- 生成可变性树
- 提供修复建议

3. 可视化

可视化：

- 打印可变性结构
- 导出为JSON
- 生成图表

答案

```
import sys
from collections.abc import Mapping, Sequence
import json
def is_mutable(obj):
    """
```

```
    判断对象是否可变
    参数：
    - obj: 要检查的对象
    返回：
    - bool: 是否可变
    """
    immutable_types = (int, float, str, bytes, bool, type(None), frozenset)
    # 基本不可变类型
    if isinstance(obj, immutable_types):
        return False
    # 元组需要递归检查
    if isinstance(obj, tuple):
        return any(is_mutable(item) for item in obj)
    # 其他都是可变的
    return True
def find_mutable_parts(obj, path="root"):
    """
    查找对象中的可变部分
    参数：
    - obj: 要分析的对象
    - path: 当前路径
    返回：
    - list: 可变部分的列表
    """
    mutable_parts = []
    if is_mutable(obj):
        mutable_parts.append({
            'path': path,
            'type': type(obj).__name__,
            'value': repr(obj)[:50]
        })
    # 递归检查容器
    if isinstance(obj, dict):
        for key, value in obj.items():
            mutable_parts.extend(find_mutable_parts(value, f"{path}[{repr(key)}]"))
    elif isinstance(obj, (list, tuple)):
        for i, item in enumerate(obj):
            mutable_parts.extend(find_mutable_parts(item, f"{path}[{i}]"))
    elif isinstance(obj, set):
        for item in obj:
            mutable_parts.extend(find_mutable_parts(item, f"{path}.item"))
    return mutable_parts
def mutability_report(obj, name="object"):
    """
    生成可变性报告
    参数：
    - obj: 要分析的对象
    - name: 对象名称
```

```
    返回：
    - dict: 详细报告
    """
    report = {
        'name': name,
        'type': type(obj).__name__,
        'is_mutable': is_mutable(obj),
        'size_bytes': sys.getsizeof(obj),
        'mutable_parts': find_mutable_parts(obj, name)
    }
    # 添加建议
    if report['mutable_parts']:
        report['suggestion'] = "对象包含可变部分，不适合作为字典键或缓存键"
    else:
        report['suggestion'] = "对象完全不可变，可以安全用作字典键"
    return report
def print_mutability_tree(obj, indent=0, name="root"):
    """
    打印可变性结构树
    参数：
    - obj: 要打印的对象
    - indent: 缩进级别
    - name: 节点名称
    """
    prefix = "  " * indent
    mutable_marker = "✓ " if is_mutable(obj) else "✗ "
    print(f"{prefix}{mutable_marker} {name}: {type(obj).__name__}")
    # 递归打印子节点
    if isinstance(obj, dict) and indent < 5:
        for key, value in list(obj.items())[:5]:
            # 限制显示数量
            print_mutability_tree(value, indent + 1, f"[{repr(key)}]")
    elif isinstance(obj, (list, tuple)) and indent < 5:
        for i, item in enumerate(obj[:5]):
            # 限制显示数量
            print_mutability_tree(item, indent + 1, f"[{i}]")
def export_report_json(report, filename):
    """
    导出报告为JSON
    参数：
    - report: 报告字典
    - filename: 文件名
    """
    with open(filename, 'w', encoding='utf-8') as f:
        json.dump(report, f, indent=2, ensure_ascii=False)
    print(f"报告已导出到{filename}")
# === 使用示例 ===
print("=== 可变性检测工具示例 ===\n")
```

```
# 示例1：检测简单对象
test_objects = {
    '整数': 42,
    '字符串': "hello",
    '列表': [1, 2, 3],
    '元组': (1, 2, 3),
    '混合元组': ([1, 2], 3),
    '字典': {'a': 1, 'b': [2, 3]},
    'frozenset': frozenset([1, 2, 3])
}
print("基本类型可变性检测:")
for name, obj in test_objects.items():
    mutable = is_mutable(obj)
    marker = "  可变" if mutable else "  不可变"
    print(f"
            {marker} {name}: {type(obj).__name__}")
# 示例2：查找可变部分
print("\n复杂对象可变部分分析:")
complex_obj = {
    'name': 'config',
    'values': [1, 2, [3, 4]],
    'settings': {
        'debug': True,
        'data': [[5, 6], [7, 8]]
    }
mutable_parts = find_mutable_parts(complex_obj)
for part in mutable_parts:
    print(f"
            {part['path']}: {part['type']}")
# 示例3：生成详细报告
print("\n详细可变性报告:")
report = mutability_report(complex_obj, "complex_obj")
print(f" 名称: {report['name']}")
print(f" 类型: {report['type']}")
print(f" 可变: {report['is_mutable']}")
print(f" 大小: {report['size_bytes']} 字节")
print(f" 可变部分数量: {len(report['mutable_parts'])}")
print(f" 建议: {report['suggestion']}")
# 示例4：可变性树
print("\n可变性结构树:")
print_mutability_tree(complex_obj)
# 示例5：对比不同数据结构
print("\n\n数据结构对比:")
structures = {
    '纯不可变': (1, 2, (3, 4), "hello"),
    '混合结构': (1, [2, 3], {'a': 4}),
    '纯可变': [1, 2, [3, 4], {'a': 5}]
}
```

```
for name, struct in structures.items():
    print(f"\n{name}:")
    print_mutability_tree(struct, name=name)
```

解析

58 • 填空 / 简答

实现防止意外修改的数据容器：

1. SafeContainer类：

- 包装列表/字典
- 所有修改返回新对象
- 原对象保持不变

2. 操作追踪

操作追踪：

- 记录所有操作
- 支持撤销/重做
- 导出操作历史

3. 性能优化

性能优化：

- 写时复制
- 结构共享
- 延迟计算

答案

```
import copy
from datetime import datetime
class SafeContainer:
    """
    安全数据容器 - 所有修改返回新对象
    特性：
    1. 写时复制（Copy-on-Write）
    2. 操作追踪
    3. 撤销/重做支持
    4. 原对象永不变
    使用示例：
    container = SafeContainer([1, 2, 3])
    new_container = container.append(4) # 返回新对象
    """
    def __init__(self, data):
        """
        初始化容器
        参数：
        - data: 初始数据（列表或字典）
        """
        self._data = copy.deepcopy(data)
        self._history = []
        self._operation_log = []
        self._redo_stack = []
    # === 列表操作 ===
    def append(self, item):
```

```
        """添加元素，返回新容器"""
        if not isinstance(self._data, list):
            raise TypeError("append只能用于列表")
        new_data = self._data.copy()
        new_data.append(item)
    return self._create_new(new_data, f"append({repr(item)})")
def extend(self, items):
    """扩展列表，返回新容器"""
    if not isinstance(self._data, list):
        raise TypeError("extend只能用于列表")
    new_data = self._data.copy()
    new_data.extend(items)
    return self._create_new(new_data, f"extend({repr(items)})")
def remove(self, item):
    """移除元素，返回新容器"""
    if not isinstance(self._data, list):
        raise TypeError("remove只能用于列表")
    new_data = self._data.copy()
    new_data.remove(item)
    return self._create_new(new_data, f"remove({repr(item)})")
# === 字典操作 ===
def set(self, key, value):
    """设置键值，返回新容器"""
    if not isinstance(self._data, dict):
        raise TypeError("set只能用于字典")
    new_data = self._data.copy()
    new_data[key] = value
    return self._create_new(new_data, f"set({repr(key)}, {repr(value)})")
def delete(self, key):
    """删除键，返回新容器"""
    if not isinstance(self._data, dict):
        raise TypeError("delete只能用于字典")
    new_data = self._data.copy()
    del new_data[key]
    return self._create_new(new_data, f"delete({repr(key)})")
# === 辅助方法 ===
def _create_new(self, new_data, operation):
    """创建新容器并记录操作"""
    new_container = SafeContainer.__new__(SafeContainer)
    new_container._data = new_data
    new_container._history = self._history + [self._data]
    new_container._operation_log = self._operation_log + [operation]
    new_container._redo_stack = []
    return new_container
def undo(self):
    """撤销最后一次操作"""
    if not self._history:
        print("没有可撤销的操作")
```

```
        return self
    new_container = SafeContainer.__new__(SafeContainer)
    new_container._data = self._history[-1]
    new_container._history = self._history[:-1]
    new_container._operation_log = self._operation_log[:-1]
    new_container._redo_stack = self._redo_stack + [self._data]
    return new_container
def redo(self):
    """重做操作"""
    if not self._redo_stack:
        print("没有可重做的操作")
        return self
    new_container = SafeContainer.__new__(SafeContainer)
    new_container._data = self._redo_stack[-1]
    new_container._history = self._history + [self._data]
    new_container._operation_log = self._operation_log + ["redo"]
    new_container._redo_stack = self._redo_stack[:-1]
    return new_container
def get_data(self):
    """获取当前数据（副本）"""
    return copy.deepcopy(self._data)
def get_history(self):
        """获取操作历史"""
        return list(zip(self._operation_log, self._history))
    def __repr__(self):
        """字符串表示"""
        return f"SafeContainer({self._data})"
    def __len__(self):
        """长度"""
        return len(self._data)
# === 使用示例 ===
print("=== 安全数据容器示例 ===\n")
# 示例1：列表操作
print("示例1：列表操作（写时复制）")
container1 = SafeContainer([1, 2, 3])
print(f"初始: {container1}")
container2 = container1.append(4)
print(f"添加4后: {container2}")
print(f"原容器不变: {container1}")
container3 = container2.extend([5, 6])
print(f"扩展后: {container3}")
# 示例2：操作链
print("\n示例2：操作链")
result = (SafeContainer([])
    .append(1)
    .append(2)
    .append(3)
    .extend([4, 5])
```

```
    .remove(2))
print(f"链式操作结果: {result}")
# 示例3：撤销/重做
print("\n示例3：撤销/重做")
c1 = SafeContainer([1, 2])
c2 = c1.append(3)
c3 = c2.append(4)
print(f"c3: {c3}")
c2_restored = c3.undo()
print(f"撤销后: {c2_restored}")
c3_restored = c2_restored.redo()
print(f"重做后: {c3_restored}")
# 示例4：操作历史
print("\n示例4：操作历史")
history = c3.get_history()
for i, (op, state) in enumerate(history):
    print(f"
            {i+1}. {op}: {state}")
# 示例5：字典操作
print("\n示例5：字典操作")
dict_container = SafeContainer({'a': 1, 'b': 2})
print(f"初始: {dict_container}")
updated = dict_container.set('c', 3)
print(f"添加c后: {updated}")
print(f"原字典不变: {dict_container}")
deleted = updated.delete('b')
print(f"删除b后: {deleted}")
```

评分说明：

- 单选题：每题1分，共30分
- 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
- 填空题：每空1分，共10分
- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：

    总分：100分

解析

59 • 填空 / 简答

实现完整的函数参数保护机制：

1. 装饰器集合

装饰器集合：

- @readonly_params ： 参数只读
- @copy_params ： 自动复制
- @validate_immutable ： 验证不可变

2. 参数分析

参数分析：

- 检测可变参数

- 生成警告
- 自动修复

3. 测试工具

测试工具：

- 参数修改检测
- 副作用追踪
- 性能影响分析

答案 （原卷为 5 选 3，未提供本题参考答案；考试时任选 3 题作答即可）

解析

60 • 填空 / 简答

实现Pandas操作的自动优化：

1. 优化规则

优化规则：

- 避免链式索引

答案 （原卷为 5 选 3，未提供本题参考答案；考试时任选 3 题作答即可）

解析